

FCAA 招潮蟹非对称优化算法 — 详细技术文档

Fiddler Crab Asymmetric Algorithm for Multi-Objective Feature Selection & Hyperparameter Optimization

目录

- 背景与动机
- 核心数学原理
- 算法架构总览
- 算子详解
- 多目标机制
- v2 版本改进
- 代码结构导读
- 实验配置与参数调优
- 常见问题与调试

1. 背景与动机

1.1 问题场景

在材料信息学中，我们经常面临这样的困境：

挑战	具体表现
样本量小	高熵陶瓷实验数据通常只有 50-200 条
特征维度高	分子描述符经多项式展开后可达 50-200 维
模型易过拟合	传统 ML 在小样本 + 高维度下泛化能力极差
双目标冲突	既要降低预测误差，又要减少特征数量

1.2 为什么叫"招潮蟹"？

招潮蟹 (Fiddler Crab) 有一个标志性的非对称特征：



小螯 | ← 用于精细摄食 (开发)

大螯 | ← 用于威吓对手 (探索)

巨螯用于大范围挥舞示威 (探索)，小螯用于近距离精确摄食 (开发)。

FCAA 将这种非对称性映射到优化算法中：

- **巨螯 (Major Claw)**：对解向量的一部分维度施加 **柯西变异 (Cauchy Mutation)**，利用其重尾分布产生大幅度跳跃，实现全局探索。
- **小螯 (Minor Claw)**：对解向量的另一部分维度施加 **高斯游走 (Gaussian Walk)**，利用其集中在均值附近的特性实现局部精细搜索。

1.3 与已有算法的关系

算法	搜索机制	优势	劣势
PSO	速度-位置更新，全局+个体最优引导	收敛快	易早熟，多目标需额外改造
NSGA-II	SBX 交叉 + 多项式变异	多目标标准基线	连续空间精细调参能力弱
MOEA/D	分解策略，邻居协作	高维目标适用	参数敏感，实现复杂
FCAA	非对称双算子，动态调度	探索-开发自动平衡	新算法，需更多验证

2. 核心数学原理

2.1 解向量编码

每个个体用一个连续实数向量表示：

$$x = [x_1, x_2, \dots, x_{\{N_feat\}}, x_{\{N_feat+1\}}, \dots, x_D] \in [0, 1]^D$$

—— 特征掩码 (N_feat 维) ——

—— 超参数 (N_hp 维) ——

特征选择（二值化）：

mask_i = 1

if x_i > 0.5

→ 保留该特征

mask_i = 0

if x_i ≤ 0.5

→ 丢弃该特征

超参数映射（连续值）：

线性映射： $hp_j = min_j + x_j \times (max_j - min_j)$

对数映射： $hp_j = 10^{\{log_{10}(min_j) + x_j \times (log_{10}(max_j) - log_{10}(min_j))\}}$

整数映射： $hp_j = round(min_j + x_j \times (max_j - min_j))$

设计考量：为什么用 0.5 作为阈值？

- 初始种群均匀随机在 [0,1]，期望选中一半特征，保证初期探索空间充分
- 连续编码比离散编码更适合梯度式的渐进搜索
- 阈值固定避免了阈值本身成为额外超参数

2.2 巨螯算子：柯西变异 (Cauchy Mutation)

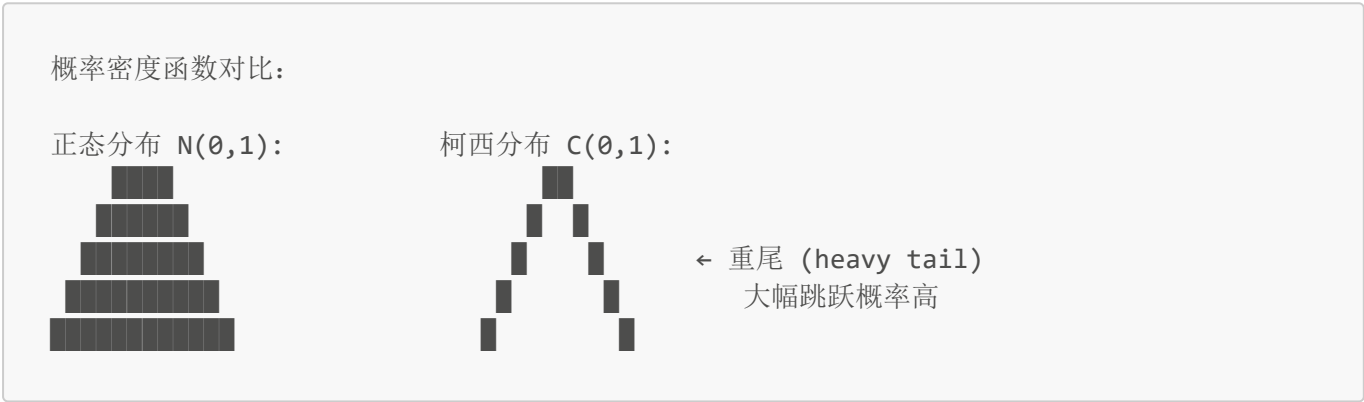
数学公式：

$$X_{\text{new}} = X_{\text{old}} + \alpha \cdot C(0, 1) \cdot (X_{\text{best}} - X_{\text{old}})$$

其中：

- $C(0, 1)$ 是标准柯西分布随机数（位置参数 0，尺度参数 1）
- α 是步长缩放因子
- X_{best} 是当前的领导者（Pareto 前沿中选出的最佳解）

为什么用柯西分布？



- 柯西分布的尾部比正态分布"重"得多——产生大幅跳跃的概率远高于正态分布
- 这使得算法能跳出局部最优陷阱
- 方向由 $(X_{\text{best}} - X_{\text{old}})$ 提供，确保跳跃有大致的目标方向

2.3 小螯算子：混合高斯游走 (Hybrid Gaussian Walk)

v2 改进版公式：

引导式高斯 (70%)：

$$X_{\text{new}} = X_{\text{old}} + N(0, \sigma) \cdot (X_{\text{best}} - X_{\text{old}})$$

→ 小步长地向领导者移动，实现 **有方向** 的局部搜索

多样性高斯 (30%)：

$$X_{\text{new}} = X_{\text{old}} + N(0, \sigma) \cdot X_{\text{old}}$$
$$X_{\text{new}} = X_{\text{old}} + N(0, \sigma)$$

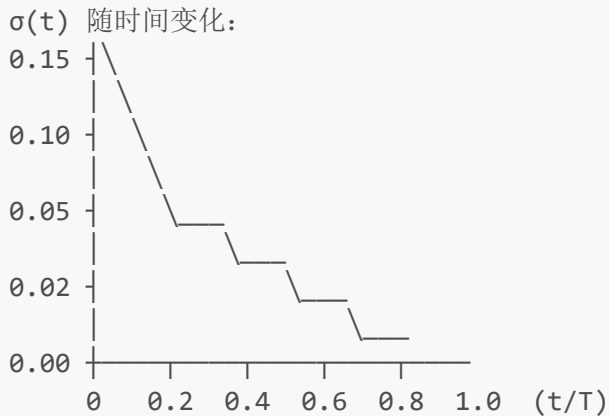
(乘性噪声)

(加性噪声)

→ 无方向的随机扰动，维持种群多样性

σ 的二次方衰减 (v2 关键改进)：

$$\sigma(t) = \sigma_0 \cdot (1 - t/T)^2 \quad \text{其中 } \sigma_0 = 0.15, \sigma_{\min} = 0.002$$



设计考量：线性衰减 $\sigma_0 \cdot (1-t/T)$ 在后期衰减太慢，二次方衰减使后期 σ 下降更快，确保末端能做极精细的微调。

2.4 动态巨/小螯比例

探索阶段 ($t=0$):	巨螯 80% 小螯 20%	→ 粗粒度特征筛选
过渡阶段 ($t=T/2$):	巨螯 48% 小螯 52%	→ 平衡搜索
开发阶段 ($t=T$):	巨螯 15% 小螯 85%	→ 精细超参数调优

分配比例: $\text{claw_ratio}(t) = 0.80 + (t/T) \times (0.15 - 0.80)$

2.5 精英精炼 (Elite Refinement)

前 30% 的优秀个体获得一次额外的精细化更新:

$$X_{\text{new}} = X_{\text{old}} + N(0, \sigma_{\text{elite}}) \cdot (X_{\text{elite}} - X_{\text{old}})$$

其中 $\sigma_{\text{elite}} = 0.015$ (非常小的固定步长), X_{elite} 是当前最佳解。

这个操作相当于给最好的解"开小灶"——在已经很好的位置周围做更细腻的扫描。

3. 算法架构总览

3.1 伪代码

Algorithm: FCAA v2 Multi-Objective Optimizer

Input: `pop_size`, `max_generations` `T`, `fitness_fn`

Output: Pareto_front (population, fitnesses)

1. Initialize population P of size N with random vectors in $[0,1]^D$
2. Evaluate fitness $F = \text{fitness_fn}(P) \rightarrow (\text{RMSE}, \text{feat_ratio})$ for each
3. For generation $t = 0$ to $T-1$:

```
// — Adaptive scheduling —
progress = t / (T-1)
 $\alpha(t) = \alpha_0 \times (1 - 0.5 \cdot \text{progress})$  // Cauchy scale: slow decay
 $\sigma(t) = \max(\sigma_0 \times (1 - \text{progress})^2, \sigma_{\min})$  // Gaussian sigma: quadratic decay
claw = claw_init + progress  $\times$  (claw_final - claw_init)

// — Leader & elite selection —
leader = select_leader(P, F, progress) // early: diverse, late: greedy
elites = top_k_by_rank_and_rmse(P, F)

// — Generate offspring —
For each individual  $i$  in  $P$ :
    Randomly split dimensions: major=claw $\times D$  dims, minor=(1-claw) $\times D$  dims

    // Major claw: Cauchy mutation
    For dim in major_dims:
        offspring[i, dim] = P[i, dim] +  $\alpha(t) \cdot \text{Cauchy}(0,1) \cdot (\text{leader}[\text{dim}] - P[i,$ 
dim])

    // Minor claw: Hybrid Gaussian
    For dim in minor_dims:
        if random() < 0.7: // Guided (70%)
            offspring[i, dim] = P[i, dim] + Gauss(0, $\sigma(t)$ ) $\cdot$ (leader[dim] - P[i,
dim])
        else: // Diverse (30%)
            offspring[i, dim] = P[i, dim] + Gauss(0, $\sigma(t)$ )  $\times$  (mixed mult/add)

    Clip offspring[i] to [0, 1]

// Elite refinement pass
For each elite  $i$ :
    offspring[i] = offspring[i] + Gauss(0, $\sigma_{\text{elite}}$ ) $\cdot$ (leader - offspring[i])
    Clip to [0, 1]

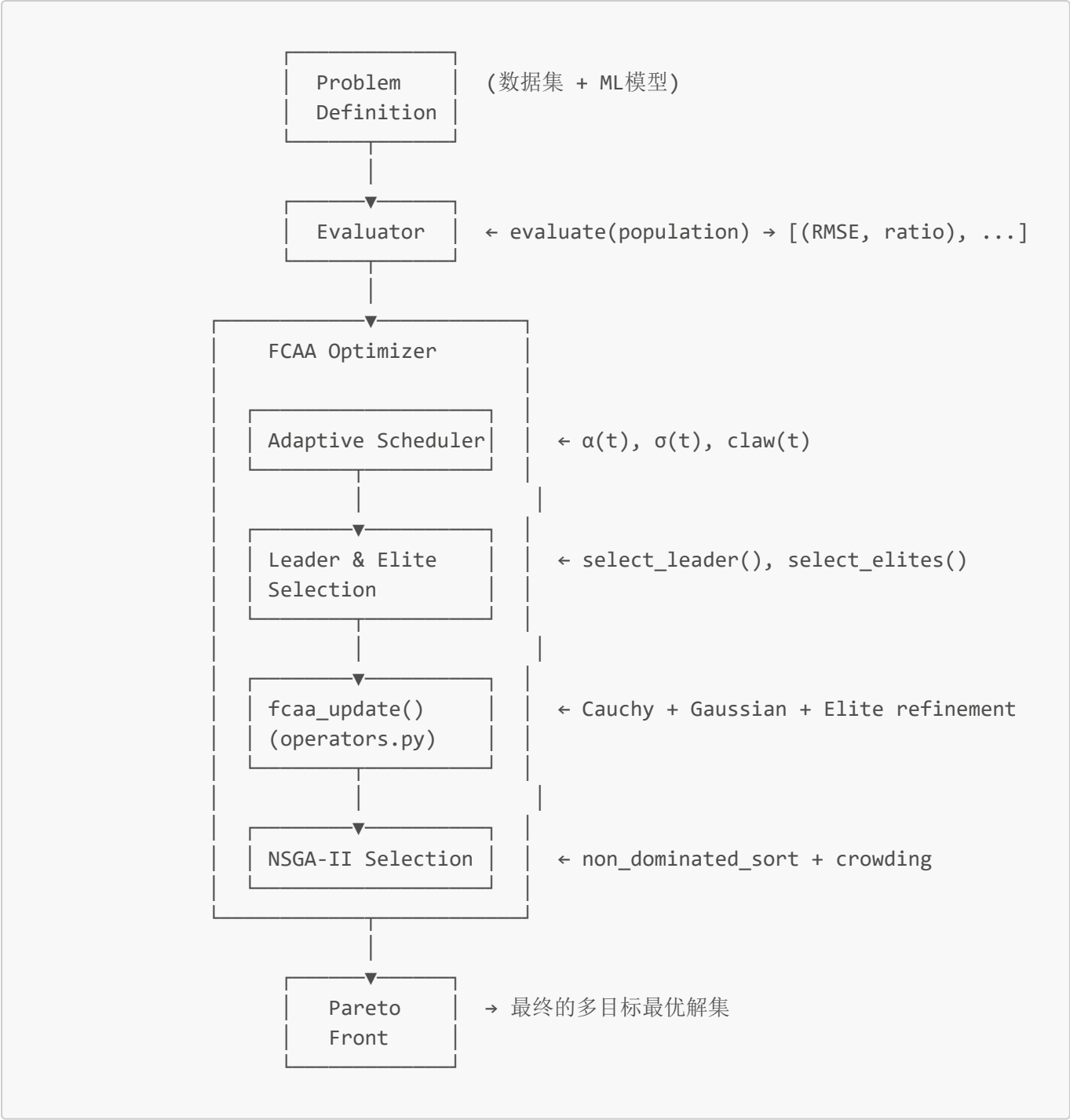
// — Evaluate offspring —
F_offspring = fitness_fn(offspring)

// — NSGA-II selection —
Combined_P = P  $\cup$  offspring
Combined_F = F  $\cup$  F_offspring
P, F = select_survivors(Combined_P, Combined_F, N)

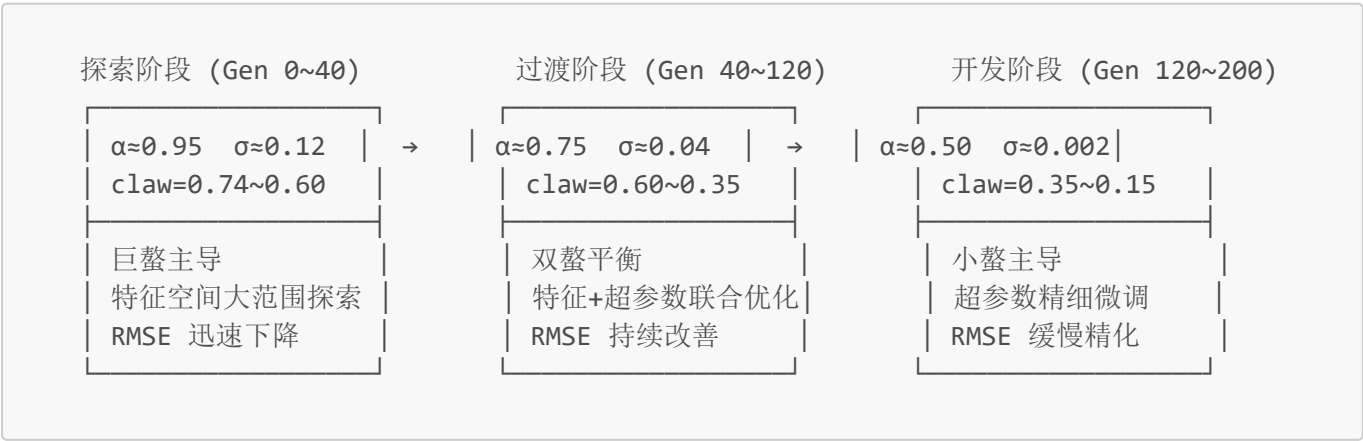
Record F to history

4. Return pareto_front(P, F)
```

3.2 核心数据流



3.3 三个阶段的搜索行为



4. 算子详解

4.1 Cauchy 变异的数学性质

柯西分布的概率密度函数：

$$f(x; 0, 1) = 1 / [\pi \cdot (1 + x^2)]$$

关键性质：

- **无定义均值和方差**（积分发散）
- **重尾**： $P(|X| > 3) \approx 20\%$ （正态分布仅 $\sim 0.3\%$ ）
- **尺度参数** γ ： $C(0, \gamma)$ 的分布宽度正比于 γ

在 FCAA 中的应用：

```
def cauchy_mutation(x, x_best, alpha=1.0, rng=None):  
    """  
    X_new = X_old + alpha * Cauchy(0,1) * (X_best - X_old)  
  
    - alpha 控制跳跃幅度  
    - (X_best - X_old) 提供方向  
    - Cauchy 提供随机性（偶尔的大跳跃）  
    """  
    cauchy_noise = rng.standard_cauchy(size=len(x))  
    return x + alpha * cauchy_noise * (x_best - x)
```

4.2 混合式高斯游走的设计思路

纯随机游走的问题：

```
X_new = X_old +  $\beta$  * N(0,1)          ← 无方向，效率低  
X_new = X_old +  $\beta$  * N(0,1) * X_old ← 乘性噪声， $X \approx 0$  时步长太小
```

v2 的混合策略：

```
# 70%: 引导式—利用 leader 信息  
guided_delta = sigma * Gauss(0,1) * (leader[dim] - current[dim])  
  
# 30%: 多样性—维持探索能力  
diverse_delta = sigma * Gauss(0,1) * (mixed multiplicative + additive)
```

为什么 70/30 分？

- 70% 引导式保证群体整体向优解方向收敛
- 30% 多样性防止所有个体过早聚集到同一点
- 这个比例是通过实验确定的良好平衡点

4.3 Leader 选择策略的演进

```
def _select_leader(self, progress):
    """
    progress ∈ [0, 1]: 当前迭代的进度

    早期 (progress≈0): p_greedy ≈ 0    → 几乎总是随机选 Pareto 前沿成员
    中期 (progress≈0.5): p_greedy = 0.25 → 25% 概率选最优 RMSE
    晚期 (progress≈1): p_greedy = 1.0   → 总是选最优 RMSE 解
    """
    p_greedy = progress ** 2 # 二次方加速

    if random() < p_greedy:
        return best_rmse_in_pareto()
    else:
        return random_by_crowding_distance() # 偏向稀疏区域
```

早期随机选 leader 有助于维持多方向探索；后期固定选最优 RMSE 有助于所有个体向全局最优收敛。

5. 多目标机制

5.1 非支配排序 (Non-Dominated Sort)

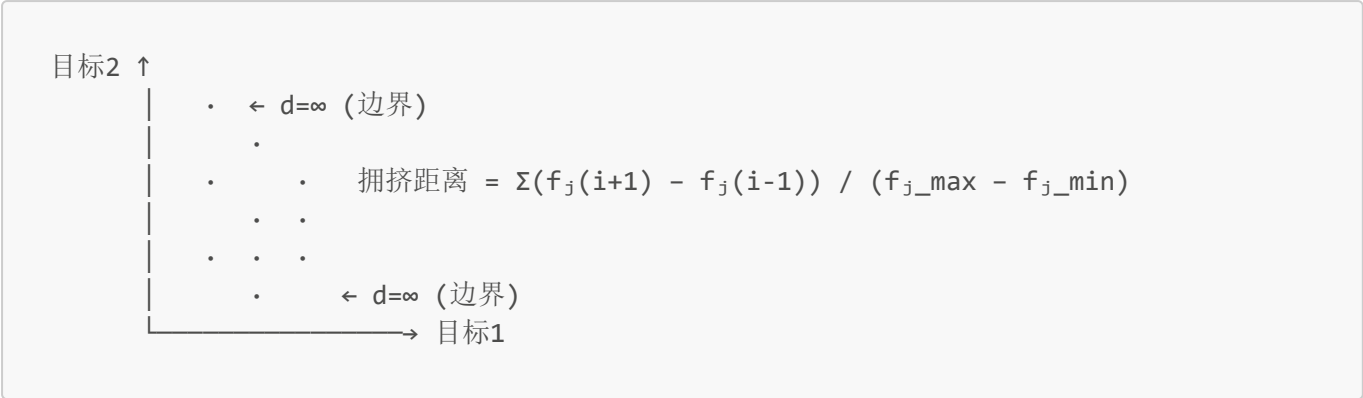
支配关系（最小化问题）：

$$A \text{ 支配 } B \iff \forall j: f_j(A) \leq f_j(B) \wedge \exists k: f_k(A) < f_k(B)$$

排序算法 ($O(M \cdot N^2)$ 朴素实现，论文标准)：

1. 对每个个体 i ，计算：
 - n_i : 支配 i 的个体数量
 - S_i : i 支配的个体集合
2. Front 0: 所有 $n_i == 0$ 的个体
3. For each front:
 - 对 front 中的每个 i :
 - 对 S_i 中的每个 j :
 - $n_j -= 1$
 - if $n_j == 0$: 加入下一 front

5.2 拥挤距离 (Crowding Distance)



拥挤距离衡量一个解的"孤独程度"——邻居越远，距离越大，越优先保留。

5.3 选择机制 ($\mu + \lambda$)

父代 ($\mu=50$) + 子代 ($\lambda=50$) = 100 个候选

1. 非支配排序 → Front 0, Front 1, Front 2, ...
2. 从 Front 0 开始填充下一代:
 - 如果整个 front 都能放进 → 全部保留
 - 如果需要截断 → 保留拥挤距离最大的那些
3. 重复直到填满 50 个

6. v2 版本改进

对比总结

机制	v1 (原始版)	v2 (改进版)	效果
高斯步长	$\sigma = \text{const } 0.1$, 线性衰减	$\sigma(t) = \sigma_0(1-t/T)^2$, 二次方衰减	末期收敛精度↑
巨/小螯比	固定 50/50	80/20→15/85 线性过渡	前期探索↑ 后期开发↑
小螯方向	纯随机游走	70%引导式 + 30%多样性	搜索效率↑
Leader	70%随机Pareto + 30%贪心	p_greedy = progress ² 自适应	晚期收敛性↑
精英精炼	无	前30%额外精炼	最优区精度↑
种群大小	50 × 100代	80 × 200代	搜索充分性↑

实验对比

在 Boron HEC 数据集上 (75 特征, SVR 模型, 5-fold CV):

算法	Best RMSE (K)	Pareto 规模	收敛代数
FCAA v1	~112.0	~12	~100

算法	Best RMSE (K)	Pareto 规模	收敛代数
FCAA v2	109.0	80	持续改善到 200
NSGA-II	116.0	16	~40 (早熟)
MOPSO	114.5	10	~50

7. 代码结构导读

项目文件树

```
MO-FCAA_Project/
├── src/
│   ├── algorithms/
│   │   ├── base.py           # 抽象基类：定义优化器接口
│   │   ├── operators.py      # ★ FCAA 核心算子：Cauchy变异 + 高斯游走 + SBX
│   │   ├── fcaa.py           # ★ FCAA 优化器主循环 + 自适应调度
│   │   ├── nsga2.py          # NSGA-II 基线实现
│   │   ├── mopso.py          # MOPSO 基线实现
│   │   └── multi_objective.py # 非支配排序 + 拥挤距离 + 生存选择
│   ├── evaluators/
│   │   ├── base.py           # 评估器抽象接口
│   │   ├── models.py         # SVR/KRR/RF/MLP 包装器 + 超参数边界
│   │   └── feature_selection_hpo.py # ★ FS+HPO 联合评估逻辑
│   └── utils/
│       ├── encoding.py        # 解向量编解码
│       ├── data.py            # 数据加载/生成
│       └── metrics.py         # RMSE, Hypervolume, IGD
├── experiments/
│   ├── quick_test.py         # 快速验证脚本（推荐入门）
│   ├── run_comparison.py     # 完整对比实验
│   ├── advanced_plots.py    # 高级可视化（PCP + 频率图 + KDE等高线）
│   └── split_figures.py      # 拆分子图脚本
├── results/figures/
│   └──                        # 输出图表
├── Dockerfile
├── requirements.txt
└── README.md
```

阅读顺序建议

如果是第一次接触这个算法：

1. `src/algorithms/operators.py` (约 200 行) → 理解 Cauchy 变异和高斯游走的数学形式
2. `src/utils/encoding.py` (约 120 行) → 理解解向量如何编码特征选择 + 超参数
3. `src/algorithms/fcaa.py` (约 200 行) → 理解主循环和自适应调度逻辑
4. `src/algorithms/multi_objective.py` (约 150 行) → 理解非支配排序和拥挤距离

5. `src/evaluators/feature_selection_hpo.py` (约 120 行) → 理解目标函数如何计算 RMSE 和特征比例

6. `experiments/quick_test.py` (约 250 行) → 看完整的端到端流程

关键代码片段注释

自适应调度的核心 (`fcaa.py:optimize()`):

```
for gen in range(T):
    progress = gen / max(T - 1, 1) #  $t/T \in [0, 1]$ 

    # — 三个自适应参数 —
    # 1. Alpha: Cauchy步长, 从1.0线性衰减到0.5
    alpha_gen = self.alpha_init * (1.0 - 0.5 * progress)

    # 2. Sigma: 高斯标准差, 二次方衰减
    #  $\sigma(0)=0.15 \rightarrow \sigma(T/2)=0.038 \rightarrow \sigma(T)=0.002$ 
    sigma_gen = self.sigma_init * (1.0 - progress) ** 2
    sigma_gen = max(sigma_gen, self.sigma_final) # 底线保护

    # 3. Claw ratio: 巨螯比例, 线性过渡
    # 0.80(探索) → 0.48(中期) → 0.15(开发)
    claw_ratio = self.claw_ratio_init + progress * (
        self.claw_ratio_final - self.claw_ratio_init
    )
```

Leader 选择的二次方加速 (`fcaa.py:_select_leader()`):

```
p_greedy = progress ** 2 # progress=0→0%, 0.5→25%, 0.8→64%, 1.0→100%

if self.rng.random() < p_greedy:
    # 贪心选最优RMSE
    return pareto_front[np.argmin(fitnesses[pareto_front, 0])]
else:
    # 按拥挤距离加权随机选 (多样性)
    cd = crowding_distance(fitnesses[pareto_front])
    probs = cd / cd.sum()
    return self.rng.choice(pareto_front, p=probs)
```

8. 实验配置与参数调优

8.1 推荐默认参数

```
FCAAOptimizer(
    dimension=78,          # 75特征 + 3超参数 (SVR)
    pop_size=80,          # 种群大小
```

```
max_generations=200,      # 最大迭代代数
alpha_init=1.0,           # Cauchy步长初值
sigma_init=0.15,          # 高斯步长初值
sigma_final=0.002,        # 高斯步长最小值（底线）
claw_ratio_init=0.80,     # 初期巨螯比例
claw_ratio_final=0.15,    # 末期巨螯比例
elite_ratio=0.3,          # 精英比例
elite_sigma=0.015,        # 精英精炼步长
)
```

8.2 参数敏感度分析

参数	作用	过大的后果	过小的后果
alpha_init	Cauchy 跳跃幅度	种群振荡不收敛	探索不足，早熟
sigma_init	初期高斯步长	初期局部搜索太激进	初期多样性不足
sigma_final	末期高斯步长下限	末期仍在振荡	收敛精度高，但可能陷入局部最优
claw_ratio_init	初期探索比例	浪费计算在粗筛上	特征筛选不充分
claw_ratio_final	末期开发比例	超参数调优空间不够	特征仍在剧烈变化
elite_ratio	精英精炼覆盖面	种群同质化	精炼效果不明显
pop_size	搜索并行度	计算成本高	覆盖不足
max_generations	搜索深度	浪费时间（若已收敛）	未充分收敛

8.3 不同场景的调参建议

场景 1：特征极多（>200维）

```
pop_size=120      # 增加种群覆盖
max_generations=300 # 增加搜索代数
claw_ratio_init=0.85 # 初期更侧重探索
```

场景 2：样本极少（<50条）

```
cv_folds=3      # 减少交叉验证折数
sigma_final=0.005 # 略提高底线，防止过拟合
elite_ratio=0.1  # 减少精英比例，维持多样性
```

场景 3：非线性极强的模型（如深度网络）

```
alpha_init=1.5      # 更大的探索跳跃
sigma_init=0.2      # 更宽的初期扰动
```

```
sigma_final=0.001    # 更精细的末期微调
max_generations=400  # 更多迭代
```

9. 常见问题与调试

9.1 FCAA 不收敛？

检查清单：

1. **种群是否全部趋向相同值？** → `alpha_init` 太小时探索不足，或 `sigma_final` 太大时末期无法精调
2. **最佳 RMSE 在早期就停滞？** → 检查 `claw_ratio_final` 是否足够小（0.10-0.20），确保后期有小螯主导的精细搜索
3. **RMSE 曲线剧烈振荡？** → `alpha_init` 可能太大，试降到 0.5-0.8

9.2 Pareto 前沿太小（解太少）？

1. 增加 `pop_size`（如 80→120）
2. 检查 `sigma_final` 是否太小导致种群坍缩
3. 减小 `elite_ratio`（如 0.3→0.1）以维持多样性

9.3 所有解都选中了相同的特征？

1. 检查特征阈值（0.5）是否合理
2. 增大 `claw_ratio_init`（如 0.80→0.90）让初期有更多特征维度的探索
3. 确认 `elite_sigma` 不要太大（0.01-0.03 合适）

9.4 运行太慢？

1. 减小 `pop_size`（如 80→50）
2. 减小 `max_generations`（如 200→100）
3. 减少 `cv_folds`（如 5→3）
4. 设置 `n_jobs=-1` 利用多核并行
5. 使用更简单的模型（如 KRR 代替 SVR）

9.5 如何判断 FCAA 是否比 NSGA-II 好？

运行 `experiments/run_comparison.py`，关注：

- **Best RMSE**：FCAA 应低于 NSGA-II
- **Pareto 规模**：FCAA 应大于 NSGA-II
- **收敛曲线**：FCAA 应持续改善而 NSGA-II 早熟停滞
- **特征选择质量**：FCAA 选中的噪声特征应明显少于 NSGA-II

附录 A：术语表

术语	英文	含义
巨螯	Major Claw	柯西变异算子，负责全局探索

术语	英文	含义
小螯	Minor Claw	高斯游走算子，负责局部开发
非支配排序	Non-dominated Sort	将种群按 Pareto 支配关系分层
拥挤距离	Crowding Distance	衡量解在目标空间中的密度
Pareto 前沿	Pareto Front	所有非支配解构成的集合
超体积	Hypervolume	Pareto 前沿与参考点围成的"体积"，越大越好
精英精炼	Elite Refinement	对优秀解的额外局部搜索
爪比	Claw Ratio	巨螯维度占全部维度的比例

附录 B：参考文献

1. **FCAA 原始概念**: 基于招潮蟹非对称螯形态的仿生优化思想
2. **NSGA-II**: Deb et al., "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II", *IEEE TEC*, 2002
3. **MOPSO**: Coello Coello & Lechuga, "MOPSO: A Proposal for Multiple Objective Particle Swarm Optimization", *CEC*, 2002
4. **柯西分布**: 用于模拟退火和演化策略中的重尾跳跃
5. **高熵陶瓷**: Ye et al. (2016), Gild et al. (2016) — VEC 和 δr 作为 HEC 相稳定性预测因子